

智能交互机器人创新实践 - 模块一：基于异步状态机的离散视觉伺服靠近方案

1. 工程背景与技术选型声明

1.1 初始需求与物理悖论

本模块的初始需求为“机器人听到呼唤后，根据声源定位并移动至人类身旁”。在实际硬件评估中，我们识别出以下致命物理约束：

1. 双足步态引发的视觉撕裂：目标设备为人形双足机器人，底盘在执行 /gaitCommand 行走时，身体会产生剧烈的钟摆式晃动，导致视觉帧产生毁灭性的运动模糊，实时动态视觉伺服在物理上不可行。
2. 多模态对齐灾难：在嘈杂的人群中，单凭声学无法精准锁定发声者的物理坐标 (Audio-Visual Alignment 失败)。

1.2 架构重构决策 (Stop-and-Go 与 交互降维)

为确保演示验收的100%成功率与系统的绝对安全，本模块做出两项核心工程妥协：

1. 摒弃“实时边走边看”，采用**“走停看”(Stop-and-Go)异步离散状态机架构**。在时间轴上严格物理隔离“视觉感知”与“步态移动”。
2. 摒弃“寻找声源”，降维为**“寻找举手姿态”**。将复杂的声学测向转化为高鲁棒性的视觉目标锁定。

2. ROS 系统接口映射规范 (I/O Mapping)

主控节点 (module1_master_node) 必须解耦所有的网络通信。建立专用的图像帧缓冲区，为后端大模型 Agent 提供高质量的视觉素材。

2.1 状态与数据侦听端 (Subscriptions - 异步非阻塞接收)

- 听觉触发源：rospy.Subscriber("/micarrays/wakeup", String, wakeup_callback)
 - 作用：被动监听唤醒信号。接收到特定字符串后，触发状态流转。
- 物理安全锁：rospy.Subscriber("/MediumSize/BodyHub/WalkingStatus", Float64, status_callback)
 - 作用：以极高频缓存机器人的底盘状态。这是决定能否开启视觉调用的绝对安全红线。
- 【新增】原始视觉缓冲：rospy.Subscriber("/chin_camera/image", Image, image_callback)
 - 作用：在内存中维护一个长度为 1 的图像队列 self.latest_frame。确保在需要“捕捉人像”时，节点内永远有一帧最新的高清物理现场快照。

2.2 视觉服务端 (Service Clients - 阻塞型请求)

- 目标锁定引擎：rospy.ServiceProxy("/ros_gesture_node/gesture_detect", GestureDetectSrv)

- 作用:传入 self.latest_frame, 阻塞等待手势检测结果, 返回手势人体边界框的坐标及尺寸。

2.3 运动控制端 (Publications - 离散指令下发)

- 视野索敌控制:rospy.Publisher("/MediumSize/BodyHub/HeadPosition", JointControlPoint)
- 离散步态控制:rospy.Publisher("/gaitCommand", Float64MultiArray)

3. 核心有限状态机流水线 (FSM Pipeline)

状态 0: 待机监视 (STATE_IDLE)

- 执行逻辑:底盘锁定, 头部居中。静默更新 self.latest_frame。
- 流转条件:唤醒标志位置为 True 时, 进入 STATE_VISION_LOCK。

状态 1: 绝对静止、视觉锁定与人像捕捉 (STATE_VISION_LOCK)

- 前置防线:检查 WalkingStatus。若为“行走中”, 立刻跳过当前帧。
- 执行逻辑:
 1. 确认安全静止后, 锁死当前的 self.latest_frame, 发起 gesture_detect 服务请求。
 2. 若未检测到目标(举手特征):下发头部舵机偏转指令(如左转 30 度), 维持本状态, 下一循环继续找。累计 3 次未找到, 抛出异常并切回 STATE_IDLE。
 3. 若检测到目标(核心目标捕捉逻辑):
 - 特征提取:获取返回的目标边界框(Bounding Box)坐标 (x, y, w, h) 。
 - 人像切片(ROI 提取):利用 OpenCV (cv2) 从刚才锁死的 self.latest_frame 矩阵中, 裁剪出该 (x, y, w, h) 对应的人体区域。将该高清切片保存至本地(如 target_captured.jpg)或推入模块三的话题队列, 完成对目标交互者的视觉捕获。
 - 空间计算:
 - 计算横向像素偏置: $\Delta x = x_{\text{center}} - x_{\text{frame_center}}$
 - 计算面积占比比率: $\text{Area_Ratio} = (w \times h) / (\text{frame_width} \times \text{frame_height})$
- 流转条件:
 - 终止条件:若 $\text{Area_Ratio} > 35\%$, 判定已到达目标身旁, 切回 STATE_IDLE。
 - 行进条件:若未达阈值, 将 Δx 映射为旋转角度, 将 $(\text{目标面积阈值} - \text{Area_Ratio})$ 映射为前进的相对步数, 状态切换至 STATE_BLIND_WALK。

状态 2: 离散盲走指令下发 (STATE_BLIND_WALK)

- 执行逻辑:将步数和转向角格式化为 Float64MultiArray, 向 /gaitCommand 发布一次性指令。发布后无条件立即切换至 STATE_WAIT_FOR_STOP。

状态 3: 步态硬核阻塞监视 (STATE_WAIT_FOR_STOP)

- 执行逻辑:机器人物理迈步中。清空上一轮的视觉缓存, 严防脏数据。主线程循环仅轮询 WalkingStatus。
- 流转条件:检测到步行状态恢复为“静止”时, 判定物理盲走结束。切回

STATE_VISION_LOCK。

4. 关键异常处理与边界防御机制

1. 指令冲突覆盖防御:在唤醒回调中加入守卫语句, 非 IDLE 状态无视新的麦克风信号。
2. 底层断联看门狗:主循环引入超时校验, 底盘状态断更超 1.0 秒立刻抛出致命错误并强制刹车。
3. 图像缓冲过期防御:在捕捉人像切片前, 必须校验 self.latest_frame 的时间戳, 若图像延迟超过 0.5 秒, 说明摄像头进程卡死, 拒绝执行识别以防撞墙。